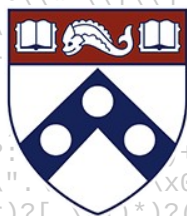


Regex

CIS 5450, Spring 2026

Harry Smith

Slides adapted from Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

- Course logistics
- Brief review
- Structured vs. unstructured data
- Regex
- Writing good notebooks

Outline

- **Course logistics**
- Brief review
- Structured vs. unstructured data
- Regex
- Writing good notebooks

Upcoming Deadlines

- HW0 due Jan 29th at 10:00pm (7 days)
- HW1 released Jan 30th at 10:00pm (8 days)
 - Due Feb 12th at 10pm
- Midterm on March 3rd in class

Tentative HW Schedule

- HW0 released Jan 15, due Jan 29
- HW1 releases Jan 30, due Feb 12
- HW2 releases Feb 13, due Feb 26
- HW3 releases between midterm & spring break, due Mar 26
- Project proposals due Mar 26
- HW4 releases Mar 27, due Apr 9
- ...the rest to be shared soon™.

Office Hours

- Office hours will be available starting next week
- Check Canvas homepage

Recitations

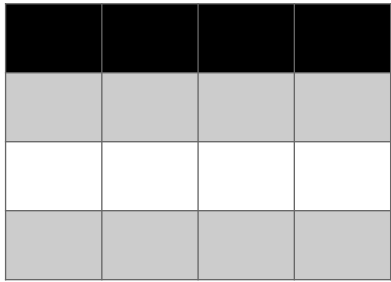
- Office hour recitation this week, Friday
- 10:45am or noon in Towne 100

Outline

- Course logistics
- **Brief review**
- Structured vs. unstructured data
- Regex
- Writing good notebooks

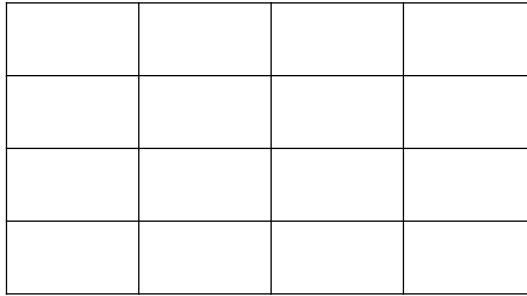
Data Modalities

- “Modality” = form, format, type, etc.



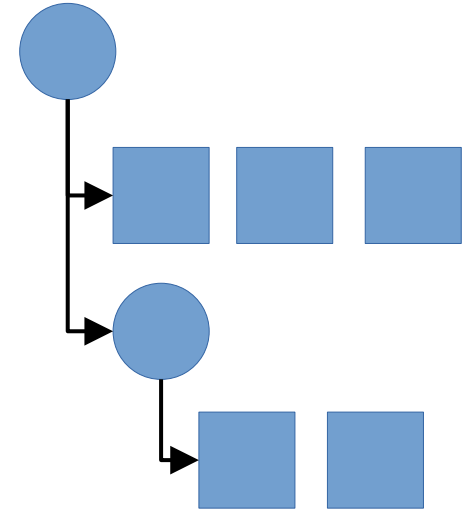
Tables

Relations
Spreadsheets
Dataframes



Arrays

Tensors
Matrix



Document

Hierarchy
Tree

Wrangling

- Our working definition:
Transforming data to be ready for analysis
- ETL:
 - **E**xtract the data
 - **T**ransform (and clean) the data
 - **L**oad the data into whatever is doing your analysis

Verb	Technique	Notes
Read data	<code>pd.read_csv()</code>	Also for json, excel, html, & more
View columns	<code>.columns</code>	—
Select columns	<code>df[["col1", "col2"]]</code>	It's indexing using a list of columns
Sort	<code>.sort_values()</code>	Pick a column to sort by, <code>ascending=False</code> for biggest first
Check types	<code>.dtypes</code>	—
String manipulations	<code>.str.*</code>	e.g. <code>.str.replace()</code> for substituting within each string

Verb	Technique	Notes
Replace cell values	<code>.replace()</code>	Works on an entire cell at a time
Change types	<code>.astype()</code>	—
Apply custom function	<code>.apply()</code>	Input is any function you provide; usually faster to find a solution with built-ins!
Check for nulls	<code>.isnull()</code>	NaN $\approx$ Null
Group & aggregate	<code>.groupby(...).*</code>	Input to groupby is the <i>key</i> for grouping
Visualizations	<code>.plot.*</code>	There's a million of these.

pandastutor.com

```
(dogs[dogs['size'] == 'medium']  
  .sort_values('type')  
  .groupby('type').median()  
)
```

	breed	type	longevity	size	weight
4	Bulldog	non-sporting	6.29	medium	45
7	Poodle	non-sporting	11.95	medium	NaN
10	Boston Terrier	non-sporting	10.92	medium	NaN
17	French Bulldog	non-sporting	9	medium	27
3	Golden Retriever	sporting	12.04	medium	60
5	Labrador Retriever	sporting	12.04	medium	67.50
6	Boxer	working	8.81	medium	NaN
15	Siberian Husky	working	12.58	medium	47.50

	breed	type	longevity	size	weight
4	Bulldog	non-sporting	6.29	medium	45
7	Poodle	non-sporting	11.95	medium	NaN
10	Boston Terrier	non-sporting	10.92	medium	NaN
17	French Bulldog	non-sporting	9	medium	27
3	Golden Retriever	sporting	12.04	medium	60
5	Labrador Retriever	sporting	12.04	medium	67.50
6	Boxer	working	8.81	medium	NaN
15	Siberian Husky	working	12.58	medium	47.50

Outline

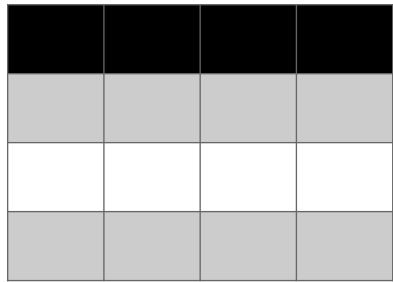
- Course logistics
- Brief review
- **Structured vs. unstructured data**
- Regex
- Writing good notebooks

Structured vs. Unstructured Data

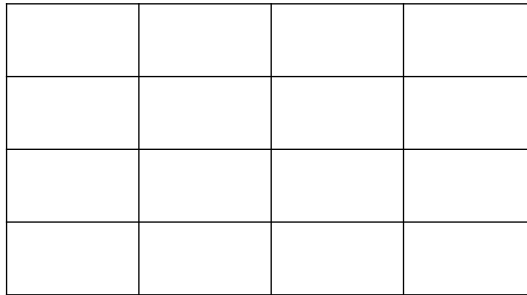
- Broad dichotomy of data
- Structured data: tables, arrays, documents
- Unstructured data: text, images, videos, audio

Structured vs. Unstructured Data

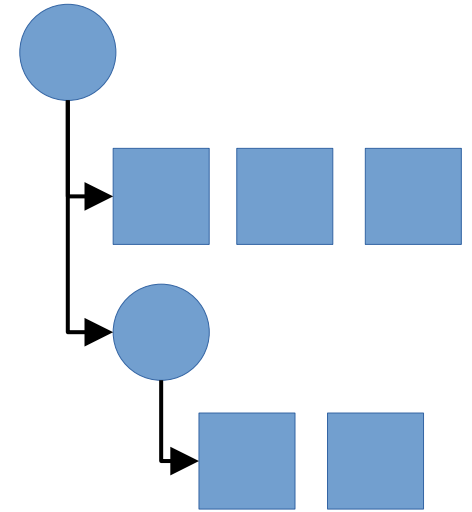
- Structured data is structured by a *schema*



Tables



Arrays

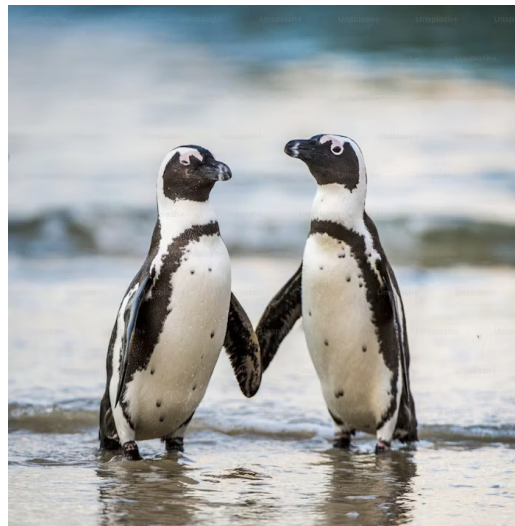


Document

Structured vs. Unstructured Data

- Unstructured data is *schema-less*

Our core insight is to model the problem of offline optimization as a matrix completion [8] problem (a technique previously used for collaborative filtering). We can represent a workload with n repeated queries and k hints (parameterizations of the query optimizer that potentially result in different query plans, as in [2, 13, 26, 30]) as a *workload matrix* $\mathbf{W}$, where each entry is the latency of a plan, as shown in Figure 1. Selecting the best hint for each query amounts to taking a row-wise minimum. Unfortunately, computing the whole matrix would require $n \times k$ query executions, which could be prohibitive, so instead we consider the workload matrix to be *partially observed*: some entries are known (observed, or part of the training set), and other entries are unknown (unobserved).



Structured vs. Unstructured Data

- Unstructured data is *schema-less*
- Why are rows / columns considered schema but paragraphs / stanzas / segments not?
- Schema must:
 - Structure **any** and **every** piece of data
 - Be **unambiguous**
 - Be **computable**

Outline

- Course logistics
- Brief review
- Structured vs. unstructured data
- **Regex**
- Writing good notebooks

Regex

- Regex = **Regular expressions**
- They're a way of extracting information from *text* strings
- (somewhat) Related to regular languages in TOC
 - $(.+)\backslash 1$
- Extremely ubiquitous in all systems
 - e.g. pandas

```
.str.replace("[^0-9]", "", regex=True)
```

Regex: Motivation

- Processing string data often involves similar steps / tasks
- Example: remove all non-numeric characters (where might we care about this?)

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

Regex: Motivation

- Processing string data often involves similar steps / tasks
- Example: remove all non-numeric characters (where might we care about this?)

```
buf = ""
for i in s:
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:
        buf += i
return buf
```

Writing this all the time is time-consuming and suboptimal

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

`re.sub("[^0-9]", "", s)`

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

re.sub ("^[0-9]", "", s)

Regular expression module

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

`re.sub("[^0-9]", "", s)`

Regular expression module

Substitute (replace)

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

re.sub("[^0-9]", "", s)

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

re.sub("[^0-9]", "", s)

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

re.sub (" [^ 0 - 9] " , "" , **s**)

Do this to string s

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Regex: Operating Modes

- Replacement (`re.sub`)
- Matching (`re.fullmatch`)
- Searching (`re.search`)
- Find all (`re.findall`)

Regex: Operating Modes

- Replacement (`re.sub`) *Replace all occurrences of a pattern with a string*
- Matching (`re.fullmatch`) *Test if full string matches a pattern*
- Searching (`re.search`) *Find the first occurrence of a pattern starting anywhere in a string*
- Find all (`re.findall`) *Find all occurrences of a pattern in a string*

Beware: `re.match()` also exists, but you rarely want that one.

Regex Computations

s = "the quick brown fox jumped over the lazy dog."

Regular expression: **p = "the q"**

`re.fullmatch(p, s)` None (no match)

`re.search(p, s)` Match (0, 5)

`re.findall(p, s)` ["the q"]

Regex Computations

s = "the quick brown fox jumped over the lazy dog."

Regular expression: **p = "the ."**

. is a wildcard – it matches any char!

`re.fullmatch(p, s)` None (no match)

`re.search(p, s)` Match (0, 5)

`re.findall(p, s)` ["the q", "the l"]

Regex Computations

s = "the quick brown fox jumped over the lazy dog."

Regular expression: **p = "^the"**

^ means the start of the string

`re.fullmatch(p, s)` None (no match)

`re.search(p, s)` Match (0, 3)

`re.findall(p, s)` ["the"]

Regex Computations

s = "the quick brown fox jumped over the lazy dog."

Regular expression: **p = "dog\$"**

\$ means the end of the string

`re.fullmatch(p, s)` None (no match)

`re.search(p, s)` None (no match)

`re.findall(p, s)` []

Regex Computations

`s = "the quick brown fox jumped over the lazy dog."`

Regular expression: `p = "dog\\. "$`

'\\' means the next character is "literal"

`p = r"dog\\. "$`

`re.fullmatch(p, s)` None (no match)

`re.search(p, s)` Match (41, 45)

`re.findall(p, s)` ["dog."]

Regex Computations

`s = "the quick brown fox jumped over the lazy dog."`

Regular expression: `p = "the.*"`

* means "as many as possible" of prev

`re.fullmatch(p, s)` Yes

`re.search(p, s)` Match (0, 45)

`re.findall(p, s)` ["the quick brown fox..."]

Regex Computations

s = "my zip code is 19146, state is PA"

Regular expression: **p = "[A-Z][A-Z]"**

[-] means "anything between"

re.fullmatch(p, s) No

re.search(p, s) Match (31, 33)

re.findall(p, s) ["PA"]

Regex Computations

s = "my zip code is 19146, state is PA"

Regular expression: **p = "[0-9]{5}"**

{#} means "exactly # times"

`re.fullmatch(p, s)` No

`re.search(p, s)` Match (15, 20)

`re.findall(p, s)` ["19146"]

Regex Computations

s = "my zip code is 87544, state is N.M."

Regular expression: **p = "[A-Z]\\\\.?[A-Z]\\\\.?"**

? means "zero or one time"

`re.fullmatch(p, s)` No

`re.search(p, s)` Match (31, 35)

`re.findall(p, s)` ["N.M."]

Regex Computations

s = "a l <be> c <de> et"

Regular expression: **p = "<.*>"**

*** means "as many as possible"**

`re.fullmatch(p, s)` No

`re.search(p, s)` Match (3, 14)

`re.findall(p, s)` ["<be> c <de>"]

Regex Computations

`s = "a l <be> c <de> et"`

Regular expression: `p = "<.*?>"`

`*?` means **"as few as needed"**

`re.fullmatch(p, s)` No

`re.search(p, s)` Match (3, 7)

`re.findall(p, s)` [`<be>`, `<de>`]

Regex Capture Groups

- Sometimes, you want to *extract* a particular chunk of text, not just find it.
- “Los Alamos, NM 87544”
- “. *?, [A-Z]{2} [0-9]{5}”
- ... will match the whole thing.

```
[2] s = "Los Alamos, NM 87544"  
    re.fullmatch(". *?, [A-Z]{2} [0-9]{5}", s)
```

```
➞ <re.Match object; span=(0, 20), match='Los Alamos, NM 87544'>
```

Regex Capture Groups

- Sometimes, you want to *extract* a particular chunk of text, not just find it.
- “Los Alamos, NM 87544”
- “(. *?), ([A-Z]{2}) ([0-9]{5})”
- ... will extract the parts we need into *groups*

✓
0s

```
[4] s = "Los Alamos, NM 87544"  
    re.fullmatch("(.*?), ([A-Z]{2}) ([0-9]{5})", s).groups()
```

➞ ('Los Alamos', 'NM', '87544')

Regex Capture Groups

- “Los Alamos, NM 87544”
- “(?P<city>.*?), (?P<state>[A-Z]{2}) (?P<zip>[0-9]{5})”
- ... will extract the groups we need, *labeled*



```
s = "Los Alamos, NM 87544"  
pattern = "(?P<city>.*?), (?P<state>[A-Z]{2}) (?P<zip>[0-9]{5})"  
re.fullmatch(pattern, s).groupdict()
```



```
{'city': 'Los Alamos', 'state': 'NM', 'zip': '87544'}
```

Regex

Operator	Description
<code>^</code>	Start of string
<code>\$</code>	End of string
<code>[A-Z]</code>	Any character between A and Z
<code>[^A-Z]</code>	Any character <i>not</i> between A-Z
<code>.</code>	Any character
<code>*</code>	As many as possible (greedy qualifier)
<code>*?</code>	As few as needed (reluctant qualifier)
<code>+</code>	At least one, and as many as possible (greedy)
<code>+</code>	At least one, but as few as needed (reluctant)
<code>(regex)</code>	Capture group
<code>(?P<name> regex)</code>	Named capture group

Character Class Shorthands

Operator	Description
<code>\w</code>	Any letter, digit, or underscore
<code>\W</code>	Any non-letter, -digit, or -underscore
<code>\s</code>	Any whitespace
<code>\S</code>	Any non-whitespace
<code>\d</code>	Any digit
<code>\D</code>	Any non-digit
<code>\b</code>	Any word boundary (transition point between a <code>\w</code> and a <code>\W</code>)

Regex Testing

- Do not attempt regular expressions without adult (e.g. regex101.com) supervision

REGULAR EXPRESSION

2 matches (228 steps, 160µs) ⓘ

`r" (?P<addr>\w+)@(?P<domain>\w+)\.(?P<tld>com|org|net)" gm`

TEST STRING

what•the•heck↵
whattheheck@gmail.com↵
what2heck@gmail.gov↵
ahhhh@hotmail.net↵

EXPLANATION

▼ r" (?P<addr>\w+)@(?P<domain>\w+)\.(?P<tld>com|org|net)" gm

▼ **Named Capture Group** addr (?P<addr>\w+)

▼ \w matches any word character (equivalent to [a-zA-Z0-9_])

+ matches the previous token between one and unlimited times, as many times as possible, giving back as needed (greedy)

MATCH INFORMATION

▼

Match 1	14-35	whattheheck@gmail.com	🔗
Group addr	14-25	whattheheck	

Outline

- Course logistics
- Brief review
- Structured vs. unstructured data
- Regex
- **Writing good notebooks**

“Good” Code

- If you ask n developers what good code is, you'll get $n+1$ distinct answers.
 - ... and almost as many books.
- Like “good” writing, what makes code “good” is subjective and context dependent.
- My answer comes from my experience



Writing good notebooks

- Code is an essay you write for two audiences
 - Audience 1: the computer
 - Audience 2: your fellow programmers
- Writing for the computer: correct, effective, efficient
- Writing for your fellow programmers: intuitive, succinct, precise

For the Computer: Correct

- “Correct” seems obvious
 - But seems to work != correct!



```
price = 3.60
tax_rate = 0.07
total = price * (1 + tax_rate)
total
```

For the Computer: Correct

- “Correct” seems obvious
 - But seems to work != correct!



```
price = 3.60  
tax_rate = 0.07  
total = price * (1 + tax_rate)  
total
```

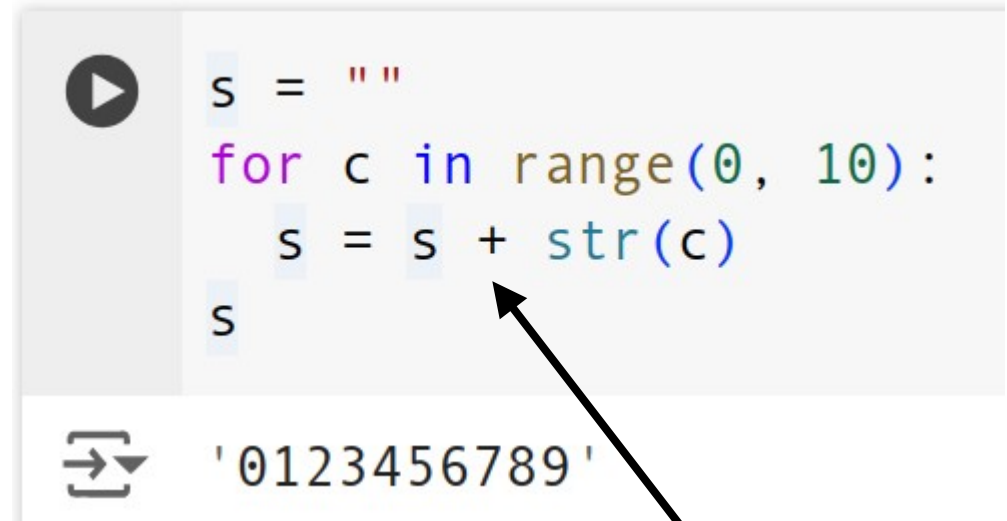


```
3.852000000000000003
```

Important to understand what is happening “under the hood”
→ And in this case, always use integer cents for money!

For the Computer: Effective

- We want code to be *fast*
 - Or, at least, predictably slow



```
s = ""  
for c in range(0, 10):  
    s = s + str(c)  
s
```

The code snippet is displayed in a light gray box with a play button icon on the left. Below the code, there is a console output showing the string '0123456789'. An arrow points from the console output to the string concatenation operation in the code.

0

01 Copy 1 character

012 Copy 2 characters

0123 Copy 3 characters

01234 Copy 4 characters

Total: 10+9+8... copies!

Python strings are *immutable*.

This code copies the string
each time!

For the Computer: Efficient

- Make good use of system resources
 - In other CS contexts, “memory leaks” or “resource leaks”

✓
0s [4] `import numpy as np`

[] `arr = np.arange(1, 1_000_000_000)`

[] `del arr`

[] `Start coding or generate with AI.`

Python 3 Google Compute Engine backend
Showing resources from 8:57 AM to
8:58 AM

System RAM
1.2 / 12.7 GB



Disk
32.8 / 107.7 GB



For the Computer: Efficient

- Make good use of system resources
 - In other CS contexts, “memory leaks” or “resource leaks”

✓ [4] `import numpy as np`

✓ 3s [5] `arr = np.arange(1, 1_000_000_000)`

▶ `del arr`

[] Start coding or [generate](#) with AI.

Python 3 Google Compute Engine backend
Showing resources from 8:57 AM to 8:59 AM

System RAM
8.6 / 12.7 GB



Disk
32.8 / 107.7 GB



For the Computer: Efficient

- Make good use of system resources
 - In other CS contexts, “memory leaks” or “resource leaks”

✓ [4] `import numpy as np`

✓ 3s [5] `arr = np.arange(1, 1_000_000_000)`

✓ 0s [6] `del arr`



Start coding or [generate](#) with AI.



Python 3 Google Compute Engine backend
Showing resources from 8:57 AM to 9:00 AM

System RAM
1.2 / 12.7 GB



Disk
32.8 / 107.7 GB



For other Humans: Intuitive

- Code is the end result of a process – report the result, but make the intuition clear!

```
def compute_point(a, b, c):  
    a %= 12  
    a /= 12  
    a += (b / 60) + (c / 60*60)  
    a *= math.pi * 2  
    return (math.cos(a), math.sin(a))
```

```
def compute_hour_hand_position(hrs, mins, secs):  
    theta = hrs % 12  
    theta /= 12  
    theta += (mins / 60) + (secs / 60*60)  
    theta *= math.pi * 2 # convert to radians  
  
    # Compute x and y coordinates from the angle.  
    return (math.cos(theta), math.sin(theta))
```

For other Humans: Intuitive

- Code is the end result of a process – report the result, but make the intuition clear!

```
def compute_point(a, b, c):  
    a %= 12  
    a /= 12  
    a += (b / 60) + (c / 60*60)  
    a *= math.pi * 2  
    return (math.cos(a), math.sin(a))
```

```
def compute_hour_hand_position(hrs, mins, secs):  
    theta = hrs % 12  
    theta /= 12  
    theta += (mins / 60) + (secs / 60*60)  
    theta *= math.pi * 2 # convert to radians  
  
    # Compute x and y coordinates from the angle.  
    return (math.cos(theta), math.sin(theta))
```

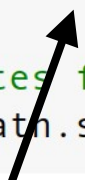
Cheat code #1: names

For other Humans: Intuitive

- Code is the end result of a process – report the result, but make the intuition clear!

```
def compute_point(a, b, c):  
    a %= 12  
    a /= 12  
    a += (b / 60) + (c / 60*60)  
    a *= math.pi * 2  
    return (math.cos(a), math.sin(a))
```

```
def compute_hour_hand_position(hrs, mins, secs):  
    theta = hrs % 12  
    theta /= 12  
    theta += (mins / 60) + (secs / 60*60)  
    theta *= math.pi * 2 # convert to radians  
  
    # Compute x and y coordinates from the angle.  
    return (math.cos(theta), math.sin(theta))
```



Cheat code #2: comments

For other Humans: Intuitive

- Code is the end result of a process – report the result, but make the intuition clear!

✓ Helper functions

We'll first define a few helper functions to compute the x and y coordinates of our clock.

```
✓ [9] def compute_hour_hand_position(hrs, mins, secs):  
  0s  ⚡ theta = hrs % 12  
      theta /= 12  
      theta += (mins / 60) + (secs / 60*60)  
      theta *= math.pi * 2 # convert to radians  
  
      # Compute x and y coordinates from the angle.  
      return (math.cos(theta), math.sin(theta))
```

Cheat code #3: text blocks

For other Humans: Succinct

- Don't be verbose.
Nobody wants to read that much.
 - Makes ChatGPT an especially bad comment-writer...
 - But not just about comments!

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) > 2:
        for c in s:
            if c >= 'A':
                if c <= 'Z':
                    # valid, check next char
                    continue
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False

    return True
```

For other Humans: Succinct

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) > 2:
        for c in s:
            if c >= 'A':
                if c <= 'Z':
                    # valid, check next char
                    continue
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False

    return True
```

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) <= 2:
        return False

    for c in s:
        if c < 'A':
            return False

        if c > 'Z':
            return False

    return True
```

Improvement 1: negate conditions

For other Humans: Succinct

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) <= 2:
        return False

    for c in s:
        if c < 'A':
            return False

        if c > 'Z':
            return False

    return True
```

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) <= 2:
        return False

    for c in s:
        if c < 'A' or c > 'Z':
            return False

    return True
```

Improvement 2: combine logical conditions

For other Humans: Succinct

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) <= 2:
        return False

    for c in s:
        if c < 'A' or c > 'Z':
            return False

    return True
```

```
# check: more than 2 characters and all
# letters between A and Z
def validate_str(s):
    if len(s) <= 2:
        return False

    return all(c >= 'A' or c <= 'Z' for c in s)
```

Improvement(?) 3: use built-ins for common patterns

For other Humans: Succinct

```
# check: more than 2 characters and all  
# letters between A and Z
```

```
def validate_str(s):  
    if len(s) <= 2:  
        return False  
    return all(c >= 'A' or c <= 'Z' for c in s)
```

```
[1]: # check: more than 2 characters and all  
# letters between A and Z  
def validate_str(s):  
    return re.fullmatch('[A-Z][A-Z][A-Z]+$', s) is not None
```

Improvement 4: use regex

For other Humans: Precise

- Don't just convey some vague sense of what is happening – convey with precision!

```
def v(h):  
    return np.sqrt(2.0 * 9.8 * h)
```

```
GRAVITY = 9.8  
def velocity(height):  
    # Compute v using the conservation of energy formula:  
    #     mass * gravity * height = (1/2) mass * velocity^2  
    # Mass cancels. Solve for velocity:  
    #     sqrt(2 * gravity * height) = velocity  
  
    return np.sqrt(2.0 * GRAVITY * h)
```

```
def velocity(height):  
    # convert height to velocity when energy is conserved  
    return np.sqrt(2.0 * 9.8 * h)
```

As always, depends on your audience!

“Good” Code

Correct. The code should produce the correct output.

Effective. the code should run quickly.

Efficient: the code should minimize resource usage.



Intuitive: the code should convey the intuition behind it.

Succinct: the code should be short.

Precise: the code should show humans what it is doing.

My Super-Good Paper

Lozem ipsum dolor sit amet, est gracco epicuri nadipting ex, id perius accumsan vel. Ne singulis mensuratum vis. Errem eligendi inductum ne his, nobis dicunt id pri, nec tation molles ei. Te videri dolorem vim, ne dolor quoque appellantur eos. Ne modus dicit complexitur vis, has ex evertitur referrentur. Sea ipsum tritani ea, velit indicabit ne sea.

Ex vim munere inimicus reprimitque. Cu dui accumsan abhorrant vel, his ex nihil repudiandus contentiones. Dicta tempus pri ne, erit elero vivendo ne sea, vim hinc liberavisse definitionem in. Sit omnesque inaudia determinat ex, per at honorum deseruisse, elis homero reprehendat duo in.

Veni determinat ex qui, at vis ullam partem albuscia. Liber laboris vim ea. Te has eret atqui putent. Est solent molestie eripidis te, namo debet dolor eos ea.

At ceteros indicabit per. Ex est saepe bonorum. Sea inani harum erit ea, has ridens ceteros ea, ea phaedrum quacundam mel. His id hinc nullam facilis, neque invenit nec te, ea sensibus pericula salutandi sed. An lorem ornatus nadipting ius. Choro scripserit his te, at cetero debitis nusquam nec. Ne dicam nominati pri, at commode platonem nam, nulla denique consequat ei eam.

Fuissent maxime ne quo, amet sonet vis ea, sed adhuc molis at. Ea stat idque vim, at neque perius moderatus mel. Venit apparuit theophrastus ea mei, probo elero venit ad vel, modus scripserit ex vis. Ea vis audire frugit pererant.

Pri putant expetendis scripserim ad, solent gloriatur duo ne, ad nobiscum platonem scribentur pri. Sit erroribus definitiones per ei, mundi inoleum pri ea, ea etiam eripuit per. Frugit dolorem significaturque id eos, aliquip efficiatur vel ea, ne eripuit eadem ius. Ex quo iusto utroque. Et officis periculis mea, possit oportet complexitur has id.

Recteque dissensio ea duo, constituto persequitur liberavisse ne sed. Homero eripuit nadipting ea eam, in qui accusata expitit, ea dicit vituperantibus eam. Per te nonnulli discere ornatus, vim in quo dicit assensio, ne definitiones disputando pri. Duo oportet explicari ei, quo te dispositioni comprehendam. Ex his scripserim necessitatibus, nonnulli impedit delicata mei ut.

Et enim tantus doctus sit. Est ne hinc volumus voluptaria. Aliquip periculis nec ea. Cum at utroque accusamus, elis assensioit eloquentiam ea mel. Eam quoniam eret ut, namum probatus interesset ne qui. Oportet gubernari ad his, te elis determinat his.

Maret inaudia ea pro. Erant sonet everti id eam, at esse determinat his, per debet videri debitis ea. His quem molles expetendis ei, errem solit ei eam. Nam delicata nadipting ad. Et equidem facilis persequitur sit, eos ea facit accusam.

Impetus maxime appellantur in nam. Omnis eligendi salutandi ei est, duo ipsum neque pertinacia id. At mel saepe laboris percipit, ne sea tale ridens percipitur. Sea liber sapient ea, nam diem meritum omnesque ea, fuisset quacundam sea te.

My Super-Good Paper

Lozem ipsum dolor sit amet, est gracco epicuri nadipting ex, id perius accumsan vel. Ne singulis mensuratum vis. Errem eligendi inductum ne his, nobis dicunt id pri, nec tation molles ei. Te videri dolorem vim, ne dolor quoque appellantur eos. Ne modus dicit complexitur vis, has ex evertitur referrentur. Sea ipsum tritani ea, velit indicabit ne sea.

Ex vim munere inimicus reprimitque. Cu dui accumsan abhorrant vel, his ex nihil repudiandus contentiones. Dicta tempus pri ne, erit elero vivendo ne sea, vim hinc liberavisse definitionem in. Sit omnesque inaudia determinat ex, per at honorum deseruisse, elis homero reprehendat duo in.

Veni determinat ex qui, at vis ullam partem albuscia. Liber laboris vim ea. Te has eret atqui putent. Est solent molestie eripidis te, namo debet dolor eos ea.

At ceteros indicabit per. Ex est saepe bonorum. Sea inani harum erit ea, has ridens ceteros ea, ea phaedrum quacundam mel. His id hinc nullam facilis, neque invenit nec te, ea sensibus pericula salutandi sed. An lorem ornatus nadipting ius. Choro scripserit his te, at cetero debitis nusquam nec. Ne dicam nominati pri, at commode platonem nam, nulla denique consequat ei eam.

Fuissent maxime ne quo, amet sonet vis ea, sed adhuc molis at. Ea stat idque vim, at neque perius moderatus mel. Venit apparuit theophrastus ea mei, probo elero venit ad vel, modus scripserit ex vis. Ea vis audire frugit pererant.

Pri putant expetendis scripserim ad, solent gloriatur duo ne, ad nobiscum platonem scribentur pri. Sit erroribus definitiones per ei, mundi inoleum pri ea, ea etiam eripuit per. Frugit dolorem significaturque id eos, aliquip efficiatur vel ea, ne eripuit eadem ius. Ex quo iusto utroque. Et officis periculis mea, possit oportet complexitur has id.

Recteque dissensio ea duo, constituto persequitur liberavisse ne sed. Homero eripuit nadipting ea eam, in qui accusata expitit, ea dicit vituperantibus eam. Per te nonnulli discere ornatus, vim in quo dicit assensio, ne definitiones disputando pri. Duo oportet explicari ei, quo te dispositioni comprehendam. Ex his scripserim necessitatibus, nonnulli impedit delicata mei ut.

Et enim tantus doctus sit. Est ne hinc volumus voluptaria. Aliquip periculis nec ea. Cum at utroque accusamus, elis assensioit eloquentiam ea mel. Eam quoniam eret ut, namum probatus interesset ne qui. Oportet gubernari ad his, te elis determinat his.

Maret inaudia ea pro. Erant sonet everti id eam, at esse determinat his, per debet videri debitis ea. His quem molles expetendis ei, errem solit ei eam. Nam delicata nadipting ad. Et equidem facilis persequitur sit, eos ea facit accusam.

Impetus maxime appellantur in nam. Omnis eligendi salutandi ei est, duo ipsum neque pertinacia id. At mel saepe laboris percipit, ne sea tale ridens percipitur. Sea liber sapient ea, nam diem meritum omnesque ea, fuisset quacundam sea te.

In this class

- Your audience are experienced data scientists
- Practice the conventions we use
 - ... because we think they are pretty good
 - ... because it'll teach you to learn conventions!
- HW1 will not consider code quality. HW2+ will.

Doing more...

Linked on Canvas:

- My (extended) tips and tricks for “clean code”



(Reading) "Good" code guide

- Zack's video on using NLTK to extract named entities from code



(Video) Extracting data from text with NLTK

Doing more...

- The Regex RPG



Slash/Escape: The Regex Adventure

